

Visão geral do que vamos construir

- Uma **classe** **Produto** que encapsula nome, estoque inicial, vendas, estoque final e situação.
 - Métodos para:
 - ler entradas (com validação),
 - calcular estoque final,
 - verificar situação,
 - exibir relatório.
 - Uma **função** **main()** que cria um objeto **Produto** e coordena a execução.
 - Explicações de POO a cada passo.
-

Instruções para o Colab

1. Abra um notebook novo.
 2. Cada “Bloco” abaixo corresponde a **uma célula**.
 3. Cole o código do Bloco 1 e execute. Teste conforme instruções.
 4. Cole o Bloco 2 e execute. Teste. E assim por diante.
-

BLOCO 1 — Classe: esqueleto e construtor (**__init__**)

Cole e execute:

```
class Produto:
    def __init__(self):
        # atributos do objeto - valores iniciais
```

```
self.nome = ""
self.estoque_inicial = 0
self.vendas = 0
self.estoque_final = 0
self.situacao = ""
```

O que testar agora

- Execute a célula (não há saída).
- Em uma nova célula, execute:

```
p = Produto()
print(type(p))
print(p.nome, p.estoque_inicial, p.vendas)
```

Você deve ver `<class '__main__.Produto'>` e os valores iniciais.

Explicação POO (parte 1)

- **Classe** é o molde (tipo) — aqui `Produto`.
- **Objeto/instância** é `p = Produto()` — uma entidade criada a partir da classe.
- **Atributos** são as variáveis ligadas à instância (`self.nome`, etc.).
- O método `__init__` é o **construtor**: executado automaticamente quando `Produto()` é chamado, e inicializa os atributos.

BLOCO 2 — Métodos de validação (encapsulamento de leitura)

Cole e execute abaixo da definição da classe (substitua a classe já existente pela versão completa com métodos, ou acrescente estes métodos dentro da classe — para facilitar, cole toda a versão abaixo que redefine a classe com validações internas):

```
class Produto:
    def __init__(self):
        self.nome = ""
```

```

        self.estoque_inicial = 0
        self.vendas = 0
        self.estoque_final = 0
        self.situacao = ""

# métodos "privados" de apoio (por convenção começam com _)
def _ler_string_nao_vazia(self, prompt):
    while True:
        valor = input(prompt).strip()
        if valor == "":
            print("Entrada inválida: o texto não pode ficar
vazio.")
            continue
        return valor

def _ler_int_nao_negativo(self, prompt):
    while True:
        entrada = input(prompt).strip()
        try:
            valor = int(entrada)
            if valor < 0:
                print("Digite um número inteiro maior ou igual a
0.")
                continue
            return valor
        except ValueError:
            print("Entrada inválida: digite um número inteiro.")

```

O que testar agora

- Execute a célula.
- Em nova célula, crie um objeto e chame os métodos de validação manualmente:

```

p = Produto()
nome = p._ler_string_nao_vazia("Teste nome: ")
print("Nome lido:", nome)
n = p._ler_int_nao_negativo("Teste inteiro: ")
print("Inteiro lido:", n)

```

Digite valores inválidos para ver a repetição da leitura.

Explicação POO (parte 2)

- Esses métodos têm `_` no início: **convenção** para métodos internos (encapsulamento conceitual).
 - Centralizar validação evita repetir código e mantém responsabilidades claras: leitura/validação separadas de lógica de negócio.
-

BLOCO 3 — Métodos públicos para ler dados (entrada do usuário)

Cole e execute (substitui a classe anterior, cole inteira com os métodos já presentes; o bloco abaixo redefine `Produto` com os métodos mostrados até aqui e os novos métodos de entrada):

```
class Produto:
    def __init__(self):
        self.nome = ""
        self.estoque_inicial = 0
        self.vendas = 0
        self.estoque_final = 0
        self.situacao = ""

    def _ler_string_nao_vazia(self, prompt):
        while True:
            valor = input(prompt).strip()
            if valor == "":
                print("Entrada inválida: o texto não pode ficar vazio.")
                continue
            return valor

    def _ler_int_nao_negativo(self, prompt):
        while True:
            entrada = input(prompt).strip()
            try:
                valor = int(entrada)
                if valor < 0:
                    print("Entrada inválida: o valor não pode ser negativo.")
                    continue
            except ValueError:
                print("Entrada inválida: o valor não pode ser um número.")
                continue
            return valor
```

```

        print("Digite um número inteiro maior ou igual a
0.")

        continue
    return valor
except ValueError:
    print("Entrada inválida: digite um número inteiro.")

# métodos públicos de interação
def pedir_nome(self):
    self.nome = self._ler_string_nao_vazia("Digite o nome do
produto: ")

    def pedir_estoque_inicial(self):
        self.estoque_inicial = self._ler_int_nao_negativo("Digite a
quantidade inicial em estoque: ")

    def pedir_vendas(self):
        while True:
            vendas = self._ler_int_nao_negativo("Digite quantas
unidades foram vendidas: ")
            if vendas > self.estoque_inicial:
                confirmar = input(f"As vendas ({vendas}) são maiores
que o estoque ({self.estoque_inicial}). Registrar mesmo assim?
(s/n): ").strip().lower()
                if confirmar == "s":
                    self.vendas = vendas
                    break
                else:
                    print("Digite novamente a quantidade vendida.")
                    continue
            else:
                self.vendas = vendas
                break

```

O que testar agora

- Execute a célula.
- Em nova célula:

```
p = Produto()
```

```
p.pedir_nome()
p.pedir_estoque_inicial()
p.pedir_vendas()
print(p.nome, p.estoque_inicial, p.vendas)
```

Teste cenários: vendas maiores que estoque e escolha s/n.

Explicação POO (parte 3)

- **Métodos públicos** são a interface do objeto: eles são quem o mundo externo usa.
 - `self` refere-se à instância corrente — atributos são guardados em `self` para persistência.
 - Lógica de negócios (ex.: confirmar vendas maiores que estoque) fica no objeto, não espalhada pelo programa.
-

BLOCO 4 — Métodos de processamento: calcular estoque e verificar situação

Cole e execute (redefine `Produto` incluindo todo o conteúdo anterior mais estes métodos — ou cole a versão completa atualizada):

```
class Produto:
    def __init__(self):
        self.nome = ""
        self.estoque_inicial = 0
        self.vendas = 0
        self.estoque_final = 0
        self.situacao = ""

    def _ler_string_nao_vazia(self, prompt):
        while True:
            valor = input(prompt).strip()
            if valor == "":
                print("Entrada inválida: o texto não pode ficar vazio.")
            else:
                continue
        return valor
```

```

def _ler_int_nao_negativo(self, prompt):
    while True:
        entrada = input(prompt).strip()
        try:
            valor = int(entrada)
            if valor < 0:
                print("Digite um número inteiro maior ou igual a
0.")
                continue
            return valor
        except ValueError:
            print("Entrada inválida: digite um número inteiro.")

def pedir_nome(self):
    self.nome = self._ler_string_nao_vazia("Digite o nome do
produto: ")

def pedir_estoque_inicial(self):
    self.estoque_inicial = self._ler_int_nao_negativo("Digite a
quantidade inicial em estoque: ")

def pedir_vendas(self):
    while True:
        vendas = self._ler_int_nao_negativo("Digite quantas
unidades foram vendidas: ")
        if vendas > self.estoque_inicial:
            confirmar = input(f"As vendas ({vendas}) são maiores
que o estoque ({self.estoque_inicial}). Registrar mesmo assim?
(s/n): ").strip().lower()
            if confirmar == "s":
                self.vendas = vendas
                break
            else:
                print("Digite novamente a quantidade vendida.")
                continue
        else:
            self.vendas = vendas
            break

def calcular_estoque_final(self):
    self.estoque_final = self.estoque_inicial - self.vendas

```

```

        return self.estoque_final

    def verificar_situacao(self):
        if self.estoque_final >= 10:
            self.situacao = "Estoque suficiente"
        elif self.estoque_final >= 1:
            self.situacao = "Estoque baixo"
        else:
            self.situacao = "Produto esgotado"
        return self.situacao

```

O que testar agora

- Execute a célula.
- Em nova célula:

```

p = Produto()
p.pedir_nome(); p.pedir_estoque_inicial(); p.pedir_vendas()
print("Final calculado:", p.calcular_estoque_final())
print("Situação:", p.verificar_situacao())

```

Explicação POO (parte 4)

- Métodos que modificam estado (`calcular_estoque_final`) escrevem em atributos `self`, então outras partes do objeto veem a mudança.
- `verificar_situacao` lê `self.estoque_final` e atualiza `self.situacao` — exemplo de **encapsulamento de regra de negócio** dentro da classe.

BLOCO 5 — Método de saída: exibir relatório

Cole e execute (novamente cole a versão completa da classe incluindo tudo anterior e este método):

```

class Produto:
    def __init__(self):
        self.nome = ""
        self.estoque_inicial = 0

```



```

        self.vendas = 0
        self.estoque_final = 0
        self.situacao = ""

    def _ler_string_nao_vazia(self, prompt):
        while True:
            valor = input(prompt).strip()
            if valor == "":
                print("Entrada inválida: o texto não pode ficar
vazio.")
                continue
            return valor

    def _ler_int_nao_negativo(self, prompt):
        while True:
            entrada = input(prompt).strip()
            try:
                valor = int(entrada)
                if valor < 0:
                    print("Digite um número inteiro maior ou igual a
0.")
                    continue
                return valor
            except ValueError:
                print("Entrada inválida: digite um número inteiro.")

    def pedir_nome(self):
        self.nome = self._ler_string_nao_vazia("Digite o nome do
produto: ")

    def pedir_estoque_inicial(self):
        self.estoque_inicial = self._ler_int_nao_negativo("Digite a
quantidade inicial em estoque: ")

    def pedir_vendas(self):
        while True:
            vendas = self._ler_int_nao_negativo("Digite quantas
unidades foram vendidas: ")
            if vendas > self.estoque_inicial:

```

```

        confirmar = input(f"As vendas ({vendas}) são maiores
que o estoque ({self.estoque_inicial}). Registrar mesmo assim?
(s/n): ").strip().lower()
        if confirmar == "s":
            self.vendas = vendas
            break
        else:
            print("Digite novamente a quantidade vendida.")
            continue
    else:
        self.vendas = vendas
        break

def calcular_estoque_final(self):
    self.estoque_final = self.estoque_inicial - self.vendas
    return self.estoque_final

def verificar_situacao(self):
    if self.estoque_final >= 10:
        self.situacao = "Estoque suficiente"
    elif self.estoque_final >= 1:
        self.situacao = "Estoque baixo"
    else:
        self.situacao = "Produto esgotado"
    return self.situacao

def exibir_relatorio(self):
    print("\n=== RELATÓRIO DE ESTOQUE ===")
    print(f"Produto: {self.nome}")
    print(f"Estoque inicial: {self.estoque_inicial}")
    print(f"Unidades vendidas: {self.vendas}")
    print(f"Estoque final: {self.estoque_final}")
    print(f"Situação: {self.situacao}")

```

O que testar agora

- Execute a célula.
- Em nova célula:

```
p = Produto()
```

```
p.pedir_nome(); p.pedir_estoque_inicial(); p.pedir_vendas()
p.calcular_estoque_final(); p.verificar_situacao()
p.exibir_relatorio()
```

Verifique que a saída está formatada e correta.

Explicação POO (parte 5)

- `exibir_relatorio` é um **método de apresentação**: separa a lógica de cálculo da lógica de apresentação (boa prática).
 - Mantemos todas as operações relacionadas ao produto concentradas na classe, facilitando manutenção e testes.
-

BLOCO 6 — Função `main()` e execução protegida

Cole e execute (em nova célula; não precisa redefinir a classe se já estiver definida):

```
def main():
    produto = Produto()
    produto.pedir_nome()
    produto.pedir_estoque_inicial()
    produto.pedir_vendas()
    produto.calcular_estoque_final()
    produto.verificar_situacao()
    produto.exibir_relatorio()

if __name__ == "__main__":
    main()
```

O que testar agora

- Execute a célula; o programa vai rodar e pedir entradas sequenciais.
- Teste os casos de uso listados abaixo.

Explicação POO (parte 6)

- `main()` organiza o fluxo de execução: **quem cria o objeto e em que ordem chama métodos**.
 - `if __name__ == "__main__":` garante que o `main()` só execute quando o arquivo for rodado diretamente — útil se este arquivo for importado como módulo em outro programa (boa prática em projetos).
-

Testes e casos de uso (para o aluno executar)

Peça ao aluno que rode o programa (célula com `main()`) e teste:

1. Estoque suficiente: estoque inicial 20, vendas 5 → final 15 → situação "Estoque suficiente".
 2. Estoque baixo: inicial 8, vendas 3 → final 5 → "Estoque baixo".
 3. Produto esgotado: inicial 5, vendas 5 → final 0 → "Produto esgotado".
 4. Vendas maiores que estoque: inicial 5, vendas 10 → responda `n` (corrija) e depois `s` (permite registrar).
 5. Entradas inválidas: digitar "abc" para números — o laço de validação pede novamente.
-

Boas práticas e pontos pedagógicos a destacar ao aluno

- cada método tem responsabilidade única (princípio SRP — Single Responsibility Principle);
- `self` é sempre a instância atual; atributos em `self` persistem enquanto o objeto existir;
- métodos com `_` são internos — não faça dependência externa neles;
- separar leitura/validação (`pedir_*`) de cálculo (`calcular_*`) e de apresentação (`exibir_*`) facilita testes e manutenção;

- `main()` faz a orquestração — o objeto `Produto` não decide quando o programa termina, apenas realiza suas tarefas.